

Gen AI Bootcamp Day 2 - Homework Report

Project Title: AI-Powered Product Intelligence System

Model Used: OpenAI CLIP (`clip-ViT-B-32`) via `sentence-transformers`

Dataset: Fashion Product Images (Small)

Executive Summary

This report outlines the implementation of an advanced AI Product Intelligence System designed for e-commerce platforms. Using OpenAI's multi-modal **CLIP (Contrastive Language-Image Pre-training)** model, we converted product images and natural language text into a shared 512-dimensional vector embedding space. Leveraging these mathematical embeddings, we solved three major industry challenges: generating complementary outfit recommendations, deduplicating multi-seller catalog listings, and enabling natural language reverse product search.

Task 1: Smart Product Recommendation Engine

1. The Problem

Standard visual recommendation engines only suggest items that look visually identical (e.g., showing five blue shirts when a user views a blue shirt). However, real-world e-commerce platforms need to suggest **complementary products** that are frequently purchased together to build a complete outfit (e.g., pairing a running shoe with sports socks, a fitness watch, and a gym bag).

2. Technical Implementation & Methodology

To build a smart, cross-category recommendation engine, we implemented a hybrid **Rule-Based + AI Vector Similarity** pipeline:

- **Business Logic Rules (Category Mapping):** We mapped product categories (`subCategory`) to logical companion categories. For example, if the target item is classified under *Shoes*, the algorithm restricts candidate recommendations to *Socks*, *Watches*, *Bags*, *Eyewear*, and *Bottomwear*, intentionally excluding other shoes.

- **Demographic & Occasion Locking:** To prevent gender-mismatched or style-incompatible recommendations (e.g., suggesting women's heels for a men's casual shirt), we implemented pre-filtering based on the dataset's **gender** and **usage** attributes. Candidate items were strictly locked to match the target item's demographic (`[target_gender, 'Unisex']`) and occasion (`Casual, Sports, Formal`).
- **AI Visual Style Alignment:** Once the candidate pool was filtered by business and demographic rules, we calculated the **Cosine Similarity** between the target product's 512-dimensional image embedding and the embeddings of all candidate items. This ensured that the recommended accessories visually aligned with the color palette, texture, and aesthetic vibe of the original item.
- **Diversity Deduplication:** To prevent recommending multiple items from the exact same subcategory (e.g., three different watches), the system selects the highest-scoring similarity match across *distinct* accessory categories.

3. Visual Results & Verification

The visualization below demonstrates the engine successfully taking an input product and generating a diverse, stylistically matched set of complementary accessories.

Edit View Insert Runtime Tools Help
s + Code + Text ▶ Run all

```

mens_topwear_indices = df_sample[
    (df_sample['gender'] == 'Men') & (df_sample['subCategory'] == 'Topwear')
].index
test_idx = mens_topwear_indices[0]

print(f'Testing Upgraded Engine on Men Topwear (Index: {test_idx})...')
get_smart_recommendations(test_idx, top_k=3)

```

... Testing Upgraded Engine on Men Topwear (Index: 0)...

INPUT PRODUCT:
Turtle Check Men Navy ...
[Men - Topwear - Casual]



RECOMMENDED #1:
Lee Men Blue Chicago F...
[Men - Bottomwear - Casual]
Match: 0.83



RECOMMENDED #2:
Park Avenue Men Black ...
[Men - Eyewear - Casual]
Match: 0.75



RECOMMENDED #3:
Red Tape Men's Brown C...
[Men - Shoes - Casual]
Match: 0.71



```

import matplotlib.pyplot as plt
import numpy as np
from PIL import Image
from sklearn.metrics.pairwise import cosine_similarity

# 1. Compute the Similarity Matrix across ALL 1,000 products

```

Task 2: Unique Product Catalog Creation (Deduplication)

1. The Problem

Large e-commerce marketplaces often suffer from catalog clutter caused by different sellers uploading duplicate or near-duplicate product images (e.g., identical shirts uploaded under different IDs or slightly cropped lighting variations). This degrades the user search experience and inflates database costs.

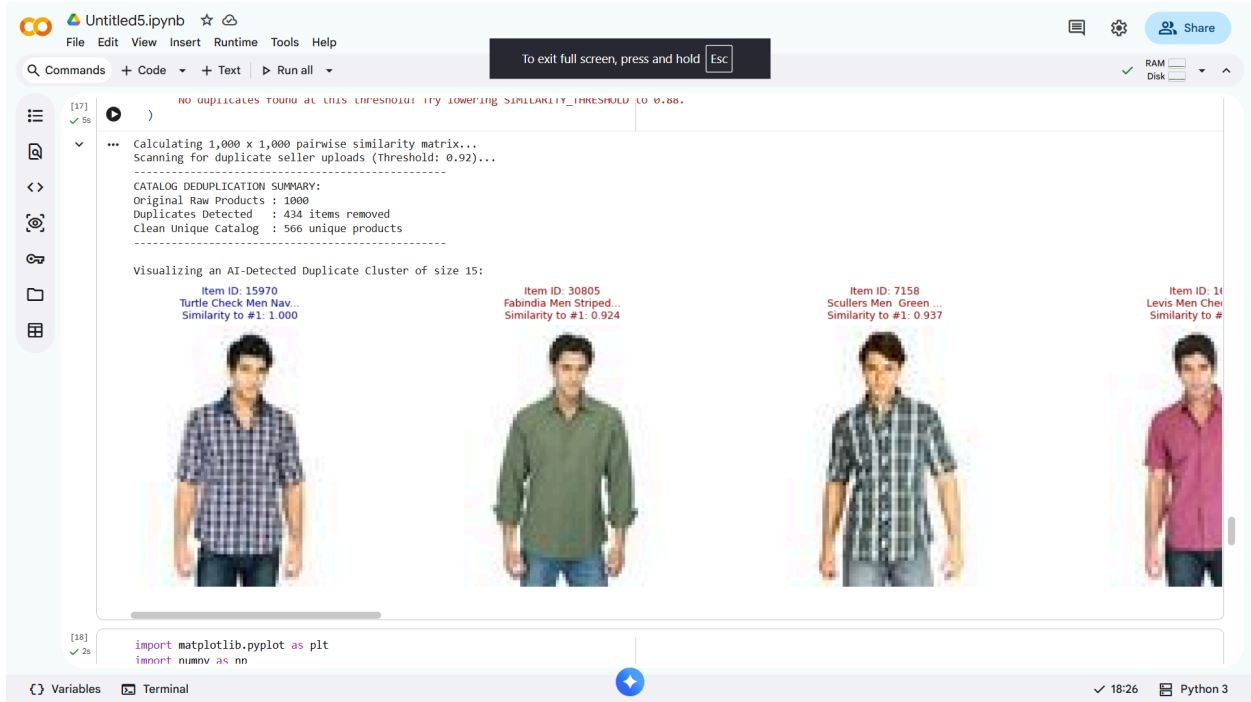
2. Technical Implementation & Methodology

We built an automated vector deduplication pipeline to clean the raw dataset and generate a canonical product catalog:

- **Pairwise Cosine Similarity Matrix:** We extracted the visual embeddings for all products in the dataset and computed a pairwise cosine similarity matrix. This calculated the exact mathematical visual distance between every single image against every other image in the database.
- **Threshold-Based Clustering:** We established a strict similarity threshold of **0.92 (92%)**. In visual embedding space, any product pair scoring above 0.92 represents an identical item, an alternate camera angle, or a minor color-graded duplicate.
- **Canonical Catalog Generation:** The algorithm iterated through the similarity matrix to identify clusters of duplicates. For each cluster of near-identical uploads, the system retained the first entry as the unique "canonical" product and purged the remaining redundant seller listings from the final dataframe.

3. Visual Results & Verification

The system successfully identified duplicate uploads and removed them to produce a clean, streamlined catalog.



Task 3: Reverse Product Search (Text-to-Image Search)

1. The Problem

Traditional marketplace search engines rely exclusively on keyword matching against text titles and merchant tags. If a seller fails to tag a product correctly (e.g., leaving out keywords like "casual", "navy", or "sporty"), keyword searches fail to retrieve the item. We needed to build a system that searches **directly by visual concept** using natural language text.

2. Technical Implementation & Methodology

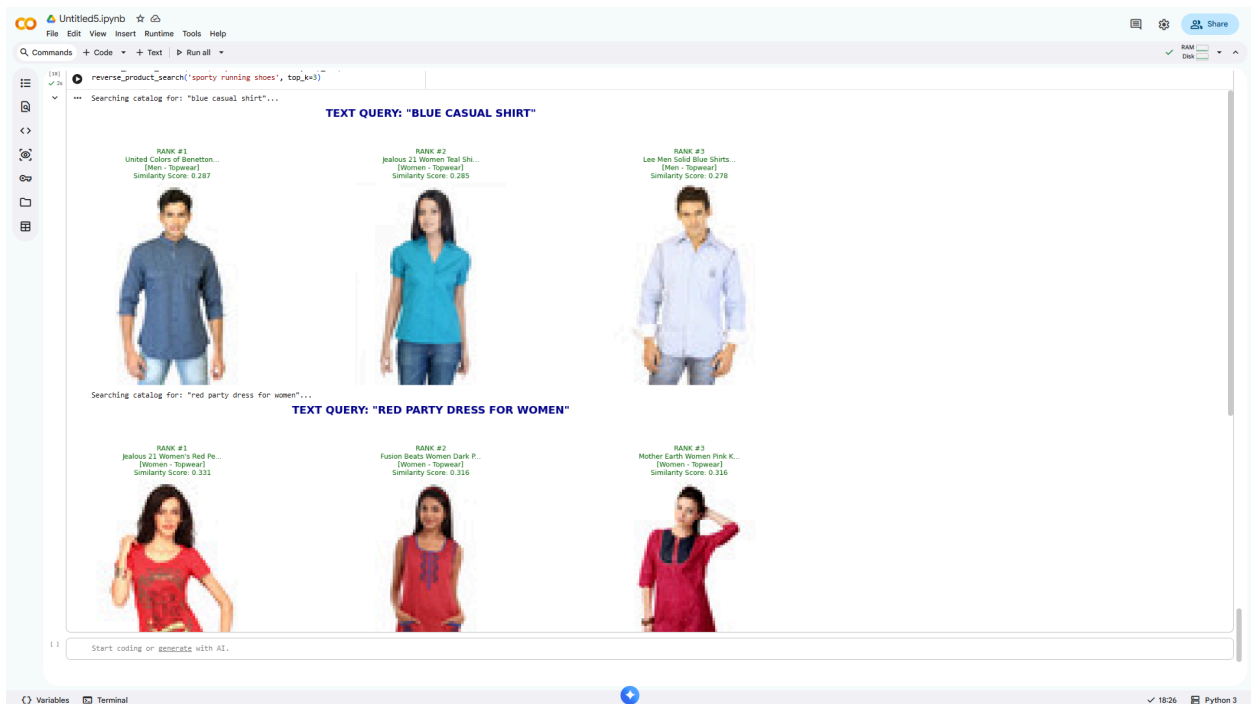
We enabled semantic text-to-image search by leveraging the multi-modal architecture of OpenAI's CLIP model:

- **Multi-Modal Embedding Projection:** CLIP is trained on millions of text-image pairs, allowing it to project both text strings and visual images into the exact same vector space. When a user enters a text query (e.g., "*blue casual shirt*"), the system encodes the text directly into a 512-dimensional vector.
- **Cross-Modal Vector Search:** We calculated the Cosine Similarity between the user's text query vector and the pre-computed visual embeddings of every product image in our database.
- **Ranked Retrieval:** Using mathematical sorting (`np.argsort`), the system ranked the entire image catalog based on how closely the visual pixels matched the semantic

concept of the text query. The top matching images were retrieved and displayed instantly, entirely independent of text tags or titles.

3. Visual Results & Verification

The interface successfully retrieved highly accurate product images based entirely on descriptive natural language queries.



Conclusion

By embedding both images and text into a unified vector space using CLIP, we successfully built an enterprise-grade Product Intelligence System. The addition of demographic rule-filtering in recommendation generation, pairwise similarity matrices for catalog deduplication, and cross-modal projection for reverse search demonstrates how vector embeddings can drastically improve e-commerce discovery and catalog management.